



Good

Evening

Everyone ☺

SUM OF ODD INDEXED ELEMENTS

Given an array of size N and Q queries with start s and end e index. For every query, return sum of all odd indexed elements from s to e .

$A[] = \{ \overset{0}{2}, \overset{1}{3}, \overset{2}{1}, \overset{3}{6}, \overset{4}{4}, \overset{5}{5} \}$

Query :

1	3	$\Rightarrow$	$A[1] + A[3] = 9$
2	5	$\Rightarrow$	$A[3] + A[5] = 11$
0	5	$\Rightarrow$	$A[1] + A[3] + A[5] = 14$
2	2	$\Rightarrow$	0

Brute Force $\Rightarrow$ TC = $O(NQ)$
SC = $O(1)$.

$A[] = \{ \overset{0}{2}, \overset{1}{3}, \overset{2}{1}, \overset{3}{6}, \overset{4}{4}, \overset{5}{5} \}$

$PSO[] = \{ 0, 2, 3, 9, 9, 14 \}$

$PSO[i] = PSO[i-1] + A[i]$ when i is odd

$PSO[i] = PSO[i-1]$ when i is even.

Quiz

Prefix sum of odd indexed elements

$i \Rightarrow$	0	1	2	<u>3</u>	4
{	2	4	3	!	5 }

PSO 2 0 4 4' 5 5 }

Code:

Function sumofOddIndexed (Array [], Queries [][], N){

PSO [N]

PSO [0] = 0

TC : $O(N+Q)$

SC : $O(N)$

for (i → 1 to N-1) {

if (i%2 == 1) {

PSO[i] = PSO[i-1] + Array[i]

}

else {

PSO[i] = PSO[i-1]

}

}

for (i → 0 to Queries.length-1) {

s = Queries[i][0]

e = Queries[i][1]

if (s == 0) {

print(PSO[e])

}

else {

print(PSO[e] - PSO[s-1])

}

}

}

SPECIAL INDEX

Given an array of size N , count the number of special index in array.

NOTE: Special Indices are those after removing which, sum of all EVEN indexed elements is equal to sum of all ODD indexed elements

count = ~~1~~ 2

$A[] = \{ \overset{0}{4} \overset{1}{3} \overset{2}{2} \overset{3}{7} \overset{4}{6} \overset{5}{-2} \}$

Ans = 2

i	A[i]	Se	So
	$\overset{0}{4} \overset{1}{3} \overset{2}{2} \overset{3}{7} \overset{4}{6} \overset{5}{-2}$		
0	{ 3 2 7 6 -2 }	8	8
1	{ 4 2 7 6 -2 }	9	8
2	{ 4 3 7 6 -2 }	9	9
3	{ 4 3 2 6 -2 }	4	9
4	{ 4 3 2 7 -2 }	4	10
5	{ 4 3 2 7 6 }	12	10

Quiz 2.

Sum of odd indices in resulting array after removal of index 2?

$A[] =$ $\{ \overset{0}{4} \overset{1}{1} \overset{2}{3} \overset{3}{7} \overset{4}{10} \}$
New $A[] = \{ 4 \ 1 \ 7 \ 10 \}$

PSO $\Rightarrow$ $\{ 0 \ 1 \ 1 \ 8 \ 8 \}$

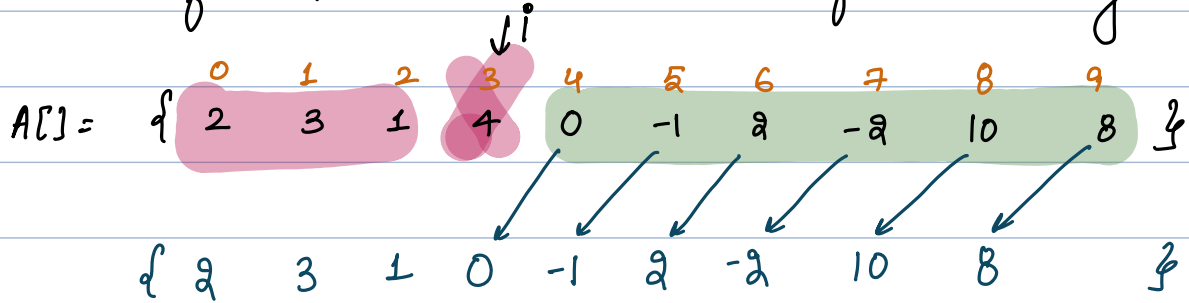
PSe $\{ 4 \ 4 \ 7 \ 7 \ 17 \}$

Sum of odd^{index} elements from 0 to $(2-1)$ ^{index}
+ Sum of even indexed elements from $(3 \text{ to } 4)$ ^{$(i+1)$}
 $1 + 10 = 11$

$\{ \overset{0}{4} \overset{1}{1} \overset{2}{7} \overset{3}{10} \}$

PSO $\Rightarrow$ $\{ 0 \ 1 \}$

Quiz 3. Sum of odd indexed elements after removing index 3?



0 to i-1. $\rightarrow$ Array did not change

	0	1	2	3	4	5	6	7	8	9
Array.	2	3	1	4	0	-1	2	-2	10	8
Even Pse	2	2	2	2	2	2	5	5	15	15
odd Pse	0	2	2	7	7	6	6	4	4	12

Sum of odd indexed element

i+1 to n-1

0 to 2.

+ Sum of even indexed element
from 4 to 9.

$$Pse[9] - Pse[3]$$

$$15 - 2 = 12$$

$$2 + 12 = \underline{14}$$

Quiz 3: Sum of even indexed elements after removing index 3?

$A[] = \{ \overset{0}{2} \overset{1}{3} \overset{2}{1} \overset{3}{4} \overset{4}{0} \overset{5}{-1} \overset{6}{2} \overset{7}{-2} \overset{8}{10} \overset{9}{8} \}$

$\{ \underset{0}{2} \underset{1}{3} \underset{2}{1} \underset{3}{0} \underset{4}{-1} \underset{5}{2} \underset{6}{-2} \underset{7}{10} \underset{8}{8} \}$

$Se \Rightarrow Pse[i-1] +$

$\overset{0}{2} \overset{1}{3} \overset{2}{1} \overset{3}{\cancel{4}} \overset{4}{0} \overset{5}{-1} \overset{6}{2} \overset{7}{-2} \overset{8}{10} \overset{9}{0} \}$

Array.

$\{ 2 \ 3 \ 1 \ 0 \ -1 \ 2 \ -2 \ 10 \ 0 \}$

Even $Pse = \{ 2 \ 2 \ 2 \ 2 \ 2 \ 5 \ 5 \ 15 \ 15 \}$

Odd $Pso = \{ 0 \ 2 \ 2 \ 7 \ 7 \ 6 \ 6 \ 4 \ 4 \ 12 \}$

Sum of even indexed elements + Sum of odd indexed elements from 4 to 9 is stored
 from 0 to 2 $2 + 12 - 7 \Rightarrow 8$

Observations

- ↳ We want to check if i is Special Index.
- ↳ Find out sum of odd indexed elements after removing i
- ↳ Find sum of even indexed element after removing i
if sum is same, it is a special index

Code:

Function count_special_index (arr[], n) {

TC : $O(N)$
SC : $O(N)$

Pse[n]

Pso[n]

// Sum of odd indexed element

for (i → 1 to N-1) {

→ Pso[0] = 0

if (i%2 == 1) {

Pso[i] = Pso[i-1] + Array[i]

}

else {

Pso[i] = Pso[i-1]

}

}

// Sum of even indexed element

for (i → 1 to N-1) {

Pse[0] = Array[0]

if (i%2 == 0) {

Pse[i] = Pse[i-1] + Array[i]

}

else {

Pse[i] = Pse[i-1]

}

}

count = 0

for (i → to n-1) {

if (i == 0)

Se = Pso[n-1] - Pso[i] // Sum from i+1 to n-1

So = Pse[n-1] - Pse[i] // Sum from i+1 to n-1.

}

else {

0 to i-1

↓

Se = Pse[i-1] + Pso[n-1] - Pso[i]

i+1 to n-1

So = Pso[i-1] + Pse[n-1] - Pse[i].

}

if (Se == So)
count++

}

print (count)

}

$\{ \begin{matrix} 0 & 1 & 2 & 3 & 4 \\ 2 & 3 & 4 & 7 & 9 \end{matrix} \}$

$P_{Se} \Rightarrow \{ 2 \quad 2 \quad 6 \quad 6 \quad 12 \}$

$S_e \Rightarrow 6 + 10 - 10 = 6$

$P_{SO} \Rightarrow \{ 0 \quad 3 \quad 3 \quad 10 \quad 10 \}$

$S_o \Rightarrow 2 + 15 - 6$

$\Rightarrow 12$

NUMBER OF EVEN NUMBERS IN GIVEN RANGE

Given an array of size N and Q queries with start (s) and end (e) index. For every query, return the sum of all Even elements from s to e .

$A \Rightarrow$ $\{ 1, 2, 3, 4, 5, 6, 7 \}$

$P[] \Rightarrow$ $\{ 0, 2, 2, 6, 6, 12, \}$

$A[]$ $\{ 12, 13, 11, 29, 28, 7 \}$

$P[] \Rightarrow$ $\{ 12, 12, 12, 12, 40, \}$

$$Pse[i] = Pse[i-1] \quad \text{if } A[i] \text{ is odd}$$

$$Pse[i] = Pse[i-1] + A[i] \quad \text{if } A[i] \text{ is even}$$

$$Sum = Pse[end] - Pse[start-1] \quad \text{if } start > 0$$

$$Sum = Pse[end] \quad \text{if } start = \underline{0}$$

code.

TC : $O(N + Q)$
SC : $O(N)$,

Pse [N]

if (Array[0] % 2 == 0)

Pse[0] = Array[0]

else

Pse[0] = 0

for (i → 1 to N-1) {

if (Array[i] % 2 == 0)

Pse[i] = Pse[i-1] + Array[i]

} else {

Pse[i] = Pse[i-1]

}

}

for (i → 0 to Queries.length-1) {

s = Queries[i][0];

e = Queries[i][1];

```
if (s == 0)
    print (pse[e])
}
else {
    print (pse[e] - pse[s-1])
}
}
```

CLOSEST MIN AND MAX

Given an array A , find the size of smallest subarrays that contains at least one occurrence of both maximum and minimum values in A .

Input:-

$A = \{ 1 \quad 3 \quad 2 \}$

$\text{max} = 3$
 $\text{min} = 1$

$\hookrightarrow \underline{2}$.

Example

$A = \{ \overset{0}{3} \overset{1}{1} \overset{2}{4} \overset{3}{2} \overset{4}{5} \overset{5}{3} \overset{6}{2} \overset{7}{4} \overset{8}{2} \overset{9}{1} \}$

$\text{Max} = 5$.

$\text{Min} = 1$

$\Rightarrow \underline{4}$.

$\text{min length} = 4$

$\begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 \\ \{ 3 & 1 & 4 & 2 & 5 & 3 \} \end{array}$

15

BRUTE FORCE APPROACH

TC : $O(N^3)$

SC : $O(1)$

function closestMinMax (A)

if (A is empty)
return 0

if (A.length == 1)
return 1

min_val = minimum value in A .
max_val = max value in A } TO DO .

n = A.length

min_length = n+1

for (i from 0 to n-1) {

for (j from i to n-1)

bool isMinpresent = false
isMaxpresent = false

for (k → i to j)

if (A[k] == min_val)
isMinpresent = true

Code:

```

    }
    if (A[k] == max_val)
        isMaxpresent = true
    }
}
if (isMinpresent &&
    isMaxpresent)
    current_length = j - i + 1
}
if (current_length < min_length)
    min_length = current_length
}
}

if min_length is still n+1
    return 0
else
    return min_length

```

OPTIMISED

APPROACH

current
length = 4

d

0	1	2	3	4	5
1	2	7	9	2	3

 Min = 1
 Max = 9
N_{min}

0	0	0	0	0	0
---	---	---	---	---	---

N_{max}

-1	-1	-1	3	3	3
----	----	----	---	---	---

last_min = 9 last_max = 7

current
= 7

0	1	2	3	4	5	6	7	8	9	10
3	<u>1</u>	4	<u>2</u>	5	3	2	<u>9</u>	2	<u>1</u>	3

 min = 1
 max = 9

 P_{min} d -1 1 1 1 1 1 1 1 1 9

 P_{max} d -1 -1 -1 -1 -1 -1 -1 7 7 7

$$9 - \min(N_{\min}[i], N_{\max}[i] + 1)$$

$$9 - \min(9, 7) + 1$$

$$9 - 7 + 1 \Rightarrow 3$$

1. Initialise two arrays of length n:

prefix_min[i] = The most recent index $\leq i$ where min_val was foundprefix_max[i] = The most recent index $\leq i$ where max_val was found

or -1 if not found yet.

2. Traverse the array once to fill these prefix arrays.

3. Then, with another pass, compute the length of smallest array ending at index i :

↳ When both $\text{prefix_min}[i]$ and $\text{prefix_max}[i]$ are not -1 , we have both values somewhere before or at index i

↳ The start of valid subarray is $\min(\text{prefix_min}[i], \text{prefix_max}[i])$

Hence $\text{length} = i - \text{start} + 1$

4. Keep track of smallest length

Function $\text{closestMinMax}(A)$ {

if A is empty
return 0

TC: $O(N)$
SC: $O(N)$

$\text{minval} \Rightarrow$ min in array
 $\text{maxval} \Rightarrow$ max in array

if $(\text{minval} == \text{maxval})$ {
 return 1
}

$n =$ length of A
 $\text{p_min}[n]$
 $\text{p_max}[n]$

initialize p_min to $[0 \text{ to } n-1] \rightarrow -1$

initialize P_max to [0 to n-1] $\rightarrow -1$

Code.

last_min_index = -1

last_max_index = -1

for (i from 0 to n-1) {

if (A[i] == min_val)
last_min_index = i

if (A[i] == max_val)
last_max_index = i

P_min[i] = last_min_index

P_max[i] = last_max_index

}
min_length = n+1

for (i from 0 to n-1) {

if (P_min[i] != -1 AND P_max[i] != -1)

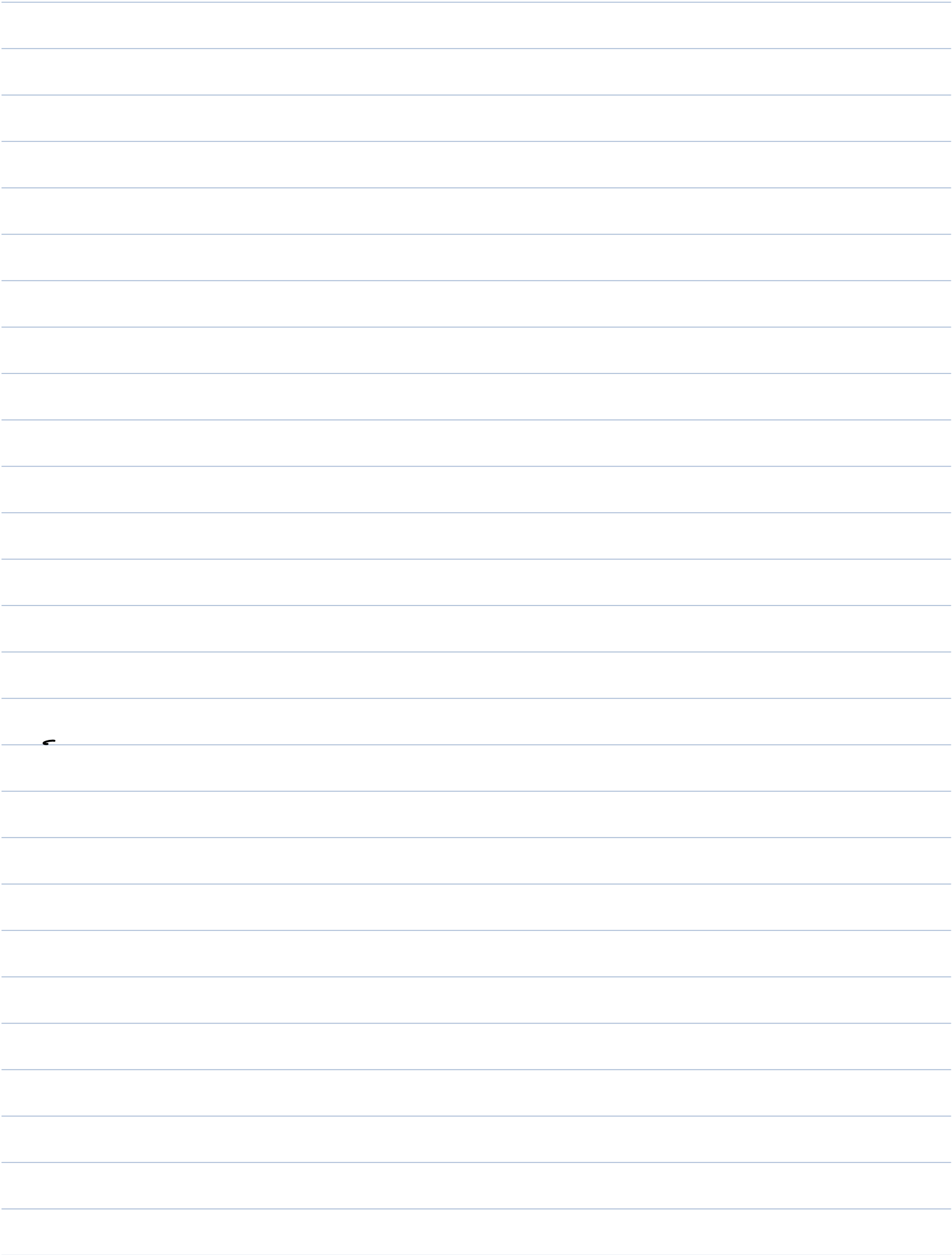
start_index = min(P_min[i], P_max[i])

current_length = i - start_index + 1

if (current_length < min_length) {

min_length = current_length

}
}
print (min_length)



min = 1
max = 5

	0	1	2	3	4	5	6	7	8	9
A = {	3	1	4	2	5	3	2	4	2	1 }

pmin {	-1	1	1	1	1	1	1	1	1	9 }
--------	----	---	---	---	---	---	---	---	---	-----

pmax {	-1	-1	-1	-1	4	4	4	4	4	4 }
--------	----	----	----	----	---	---	---	---	---	-----

$\min - l = \underline{3}$.

	0	1	2	3	4
{	1	2	3	2	1 }
<u>Pmin</u> {	0	0	0	0	4 }
<u>Pmax</u> {	-1	-1	2	2	2 }

Min = 1
Max = 3

	0	1	2	3	4	5
{	1	2	3	2	1	3 }
Pmin {	0	0	0	0	4	4 }
Pmax {	-1	-1	2	2	2	5 }

i - start + 1
2 - 0 + 1 = 3

i - start + 1
4 - 2 + 1 $\Rightarrow$ 3

start 2 end = 4

$4 - 2 + 1 = \underline{3}$ $10^6 * 10^{19} \Rightarrow$

start = 4 end = 5

$5 - 4 + 1 = \underline{2}$ $10^6 + 10^{19} \Rightarrow$

for (i = 1 to N) N

for (j = 1 to Q) Q
}

N = 1 → Q times
2 → Q times
3 → Q times
⋮
N → Q times

N Q times